🎓 Project Title

Clinic Appointment Scheduler – A Backend-Driven Patient Booking & Management System

📄 Project Description:

The **Clinic Appointment Scheduler** is a robust, backend-only application developed using **ASP.NET Core Web API** following the **3-tier architecture** and **SOLID principles**. The system is designed to streamline patient appointment scheduling and clinic operations through a secure and scalable REST API interface, tested entirely via **Postman**.

This project goes beyond traditional CRUD operations by incorporating **real-world backend functionalities** such as **email notifications**, **smart filtering**, **conflict-free scheduling**, **report uploads**, and **role-based access control** — all critical components for a modern clinical management system.

The core objective is to facilitate seamless coordination between patients and doctors while ensuring data accuracy, integrity, and maintainability. The system is flexible enough to be extended into a full-stack application in future iterations.

🔧 Key Functional Highlights:

✅ Core Entity Management (CRUD)

- **Patient Management**: Add, update, delete, and retrieve patient profiles.
- **Doctor Management**: Create and manage doctors with specializations. (Admin-restricted)
- **Appointment Management**: Book, update, cancel, and view appointments with proper status tracking (Scheduled, Cancelled, Completed).

⭐ Advanced Functionalities for A+ Grade

1. 🔔 **Email Notification System**
    - Sends **confirmation emails** upon successful appointment booking.
    - Includes a dedicated endpoint to trigger **reminder notifications** for next-day appointments.
2. 🔍 **Advanced Filtering & Search**
    - Supports filtering appointments by **doctor**, **date range**, **status**, and **partial patient name match**.
    - Includes **pagination and sorting** for scalable API access.
3. 🗓️ **Conflict-Free Appointment Scheduling**
    - Prevents double-booking by checking for overlapping time slots.

- Disallows bookings in the past and ensures time validation.
- Returns appropriate error codes (409 Conflict) when scheduling rules are violated.

4. 🧾 **Auto-Generated Appointment Summary**
   - Returns a **detailed appointment summary** (JSON format), including doctor/patient info, time, reason, and status.
   - Lays groundwork for future PDF report generation.
5. 📂 **File Upload for Medical Reports**
   - Allows uploading of medical reports or prescriptions linked to a patient.
   - Files are stored locally with metadata saved in the database.
6. 🛡️ **Role-Based Access Control (RBAC)**
   - Implements **JWT-based role checks** (Admin, Doctor, Patient).
   - Secures sensitive endpoints like doctor creation and system settings.

🧱 Architecture & Technologies Used

- **Architecture**: 3-tier (Controller → Service → Repository)
- **Framework**: ASP.NET Core Web API (.NET 6+)
- **ORM**: Entity Framework Core (Code First with Migrations)
- **Database**: SQL Server
- **Authentication**: JWT (Token-based with Roles)
- **File Handling**: IFormFile + Local Storage
- **Notification**: Email sending via mock SMTP or console logs

---

📌 Project Outcomes

By completing this project, the system provides:

- Reliable and automated appointment management
- Secure and structured data handling
- Scalable design for future UI integration
- A professional, testable backend ready for real-world deployment

Features

1. 👤 Patient Management

Endpoints:

- POST /patients – Add a new patient
- PUT /patients/{id} – Edit patient details
- DELETE /patients/{id} – Delete a patient
- GET /patients/{id} – View patient details
- GET /patients – View all patients (with optional filtering)

Description:
Allows the clinic staff to manage patients' profiles. Each patient can later be linked to appointments and reports. Fields may include name, email, phone, date of birth, address, etc.

---

2. 👨‍⚕️ Doctor Management

Endpoints:

- POST /doctors – Add a new doctor (✅ Admin only)
- PUT /doctors/{id} – Edit doctor profile
- DELETE /doctors/{id} – Remove a doctor
- GET /doctors/{id} – View doctor details
- GET /doctors – List all doctors

Description:
Doctors are assigned to appointments. Each has a specialization and availability. Only admins should be able to manage this via role-based access.

---

3. 🗓️ Appointment Scheduling

Endpoints:

- POST /appointments – Book an appointment (with conflict checking ✅)
- PUT /appointments/{id} – Update appointment info
- DELETE /appointments/{id} – Cancel an appointment
- GET /appointments/{id} – View single appointment

- GET /appointments – View list of appointments (✅ With advanced filters)

Description:
Patients book appointments with doctors. This endpoint handles appointment time, purpose/reason, and includes status: Scheduled, Completed, or Cancelled.

⭐ Advanced Features (for A+ Grade)

These go beyond CRUD and demonstrate your understanding of real-world backend systems.

---

4. 🔔 Email Notification System

Endpoints:

- POST /appointments – Triggers **confirmation email**
- POST /send-reminders – Sends **reminders for upcoming appointments**

Description:
When an appointment is booked, a confirmation email is sent (you can mock this via console logs). You also expose a /send-reminders endpoint to send email reminders for appointments happening the next day.

---

5. 🔍 Advanced Filtering & Search

Endpoint:

- GET /appointments?doctorId=&date=&status=&patientName=&page=&size=&sort=

Filters:

- Filter by Doctor ID
- Filter by Date or Date Range
- Filter by Appointment Status (Scheduled / Cancelled / Completed)
- Partial match search on Patient Name
- Pagination (page, size)
- Sorting (sort=asc / desc on date or name)

Description:
This feature allows admins or doctors to efficiently find appointments using multiple filters. Implement logic in the **Service Layer using LINQ**.

---

6. 🗓️ Conflict-Free Scheduling

Endpoint:

- POST /appointments

Logic Implemented:

- Doctor cannot have two appointments at the same time
- No booking in the past
- Check doctor's availability (if maintained)
- Return 409 Conflict if overlap found

Description:
Ensures accurate scheduling and prevents double-booking, which is critical in a clinical environment. Validations must be done in the service layer, not just controllers.

---

7. 🧾 Auto-Generated Appointment Summary

Endpoint:

- GET /appointments/{id}/summary

Output:

- JSON-formatted summary or downloadable PDF (optional)
- Includes patient name, doctor name, date/time, reason, status

Description:
This summary is useful for front-desk staff and doctors to print or archive. Even if PDF generation is not implemented, simulate it with well-structured JSON.

---

8. 📁 File Upload for Patient Reports

Endpoint:

- POST /patients/{id}/upload-report

Functionality:

- Upload PDF/image files as medical reports
- Store to /Reports folder locally
- Save file metadata in database (file path, upload date, patientId)

Description:
Adds practical document management support. Use IFormFile and MultipartFormData in .NET Core for file handling.

---

9. 🛡️ Role-Based Access Control (RBAC)

Implementation:

- Use JWT with roles (Admin, Doctor, Patient)
- Protect endpoints like POST /doctors to Admin only
- Use [Authorize(Roles = "Admin")] on restricted controllers

Description:
Implements basic security to ensure only authorized users can perform certain actions. Can simulate user login via Postman with pre-generated JWT tokens.

🔚 Summary Table of All Features

#	Feature	Endpoint(s)	Level
1	Patient CRUD	/patients, /patients/{id}	Basic
2	Doctor CRUD	/doctors, /doctors/{id}	Basic
3	Appointment CRUD	/appointments, /appointments/{id}	Basic
4	Email Notification (confirmation/reminder)	POST /appointments, POST /send-reminders	Advanced
5	Advanced Filtering/Search	GET /appointments?...	Advanced
6	Conflict-Free Scheduling	POST /appointments	Advanced
7	Appointment Summary	GET /appointments/{id}/summary	Advanced
8	File Upload for Reports	POST /patients/{id}/upload-report	Advanced
9	Role-Based Access (JWT)	All secured endpoints	Bonus ⭐